Aluna: Mirelle Casimiro Silvino

Disciplina: Sistemas Operacionais.

Ambiente utilizado: notebook com processador Intel Core i7-13620H (10 núcleos: 6 de desempenho "P", com Hyper-Threading, e 4 de eficiência "E", totalizando 16 CPUs lógicas), 16 GB de RAM e Windows 11. Os programas foram compilados com o gcc do MSYS2 UCRT64 (`gcc -Wall -O2 ... -lpthread`), que implementa a API POSIX Threads sobre as threads nativas do Windows (biblioteca *winpthreads*).

Os códigos comentados em detalhe estão nos arquivos `thread-create.c`, `thread-join.c`, `thread-print.c` e `calcpi.c`.

Observação: no Windows o tipo `long` tem 32 bits e os ponteiros têm 64 bits (modelo LLP64), por isso o `gcc -Wall` emite avisos nas conversões `(long) i` e `(void*) i` dos programas fornecidos. Como os valores são pequenos (0 a 15), o resultado não é afetado. No `calcpi.c` foi usado o tipo `intptr_t`, que tem o tamanho de um ponteiro, e o programa compila sem avisos.

1 EXERCÍCIO 1: THREAD-CREATE.C

1.1 ANÁLISE DO CÓDIGO

a) A thread `main` cria 16 threads com `pthread_create()`; cada uma executa a função `threadBody()` e recebe seu número (0 a 15) por valor, convertido para `void *`;

b) Ao contrário do `fork()`, nenhum processo novo é criado: as threads são novos fluxos de execução dentro do mesmo processo e compartilham código, variáveis globais, heap e arquivos abertos. Cada thread possui apenas sua própria pilha e seus próprios registradores;

c) Cada thread imprime `Olá`, dorme 3 s com `sleep(3)` (que suspende somente ela) e imprime `Tchau`;

d) A thread `main` não espera as demais: imprime `Main: fim` e chama `pthread_exit(NULL)`, que encerra apenas a thread main. O processo continua existindo até que a última thread termine. Se `main` usasse `return` ou `exit()`, o processo inteiro seria encerrado imediatamente e as mensagens `Tchau` nunca apareceriam.

Saída real (trechos repetidos abreviados):

Figura 1: Saída da execução de thread-create.c

```
Main: criando thread 00
Main: criando thread 01
...
Main: criando thread 15
Main: fim
t07:  Olá!
t04:  Olá!
t00:  Olá!
t01:  Olá!
t06: Olá!
...
t12: Olá!
t09: Tchau!
t11: Tchau!          <- após ~3 s
t03: Tchau!
...
t15: Tchau!
t07: Tchau!
```

1.2 A ORDEM DE CRIAÇÃO, ATIVAÇÃO E ENCERRAMENTO DAS THREADS É A MESMA?

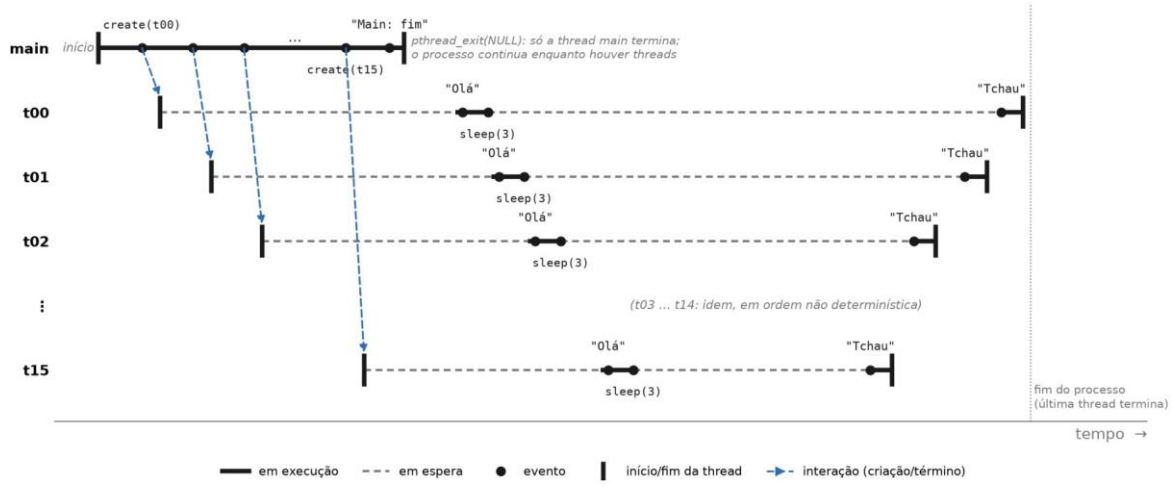
Não. A criação das threads segue a ordem do laço (00, 01, …, 15), já que é a thread main que as cria uma após a outra. A ativação (quando a thread de fato começa a rodar, o que aparece no Olá) e o encerramento (Tchau) acontecem numa sequência diferente, que varia de uma execução para outra. Na execução mostrada acima, a primeira a imprimir Olá foi a t07 e a primeira a imprimir Tchau foi a t09.

O motivo é que uma thread recém-criada apenas entra na fila de prontos. Quem escolhe o momento em que ela ganha uma CPU é o escalonador do sistema operacional, que leva em conta as CPUs livres, a carga do sistema, as interrupções e os outros processos.

Como existem várias CPUs, várias threads rodam ao mesmo tempo e a ordem entre elas não é previsível. O mesmo vale para o final: todas dormem 3 s, acordam quase juntas e a ordem em que retomam a CPU depende novamente do escalonador. O programa não tem nenhuma sincronização que imponha uma ordem. Repare também que Main: fim apareceu antes de qualquer Olá: a main terminou de criar as 16 threads e acabou antes que alguma delas fosse escalonada.

1.3 DIAGRAMA DE TEMPO

São representadas a thread main e as threads t00, t01, t02 e t15 (as demais se comportam de forma análoga). Os tempos são esquemáticos, mas a ordem dos eventos segue a execução real.



2 EXERCÍCIO 2: THREAD-JOIN.C

2.1 ANÁLISE DO CÓDIGO

a) Igual ao exercício 1, mas as threads são criadas com um objeto de atributos attr, configurado como PTHREAD_CREATE_JOINABLE;

b) Após criar as 16 threads, a thread mainchama pthread_join(thread[i], NULL) para cada uma, em ordem (00 a 15). Cada join bloqueia mainaté que a thread correspondente termine;

c) Somente após todos os joins mainimprime Main: fim, libera attrcom pthread_attr_destroy()e termina.

Saída real (trechos repetidos abreviados):

```
Main: criando thread 00
...
Main: criando thread 15
Main: aguardando thread 00      <- main bloqueia aqui
t00: Olá!
t04: Olá!
...
t15: Olá!
t15: Tchau!                     <- após ~3 s
t14: Tchau!
...
t00: Tchau!                     <- t00 termina: join(t00) retorna
t01: Tchau!
Main: aguardando thread 01      <- as demais já terminaram:
Main: aguardando thread 02         cada join retorna na hora
...
Main: aguardando thread 15
Main: fim                       <- sempre a última linha
```

2.2 OBJETIVO DO PARÂMETRO ATTR E DA CHAMADA PTHREAD_JOIN

O parâmetro attr (tipo pthread_attr_t) é um objeto que reúne os atributos de criação da thread e é passado como segundo parâmetro de pthread_create() (NULL indica atributos padrão). Com ele é possível definir o estado de desacoplamento (*detach state*), o tamanho e o endereço da pilha, a política e a prioridade de escalonamento e o escopo de disputa, entre outros. Neste programa, ele é usado para criar as threads como joinable (PTHREAD_CREATE_JOINABLE), ou seja, threads cujo término pode ser aguardado por outra thread.

O padrão POSIX já cria threads joinable por omissão, então o atributo torna essa intenção explícita (e garante o comportamento em implementações antigas). A alternativa é

PTHREAD_CREATE_DETACHED: uma thread *detached* libera seus recursos sozinha ao terminar e não pode ser aguardada.

A chamada pthread_join(t, &ret) faz a thread chamadora esperar o término da thread t, de forma análoga ao wait() para processos; se t já terminou, a chamada retorna imediatamente. Ela tem três funções:

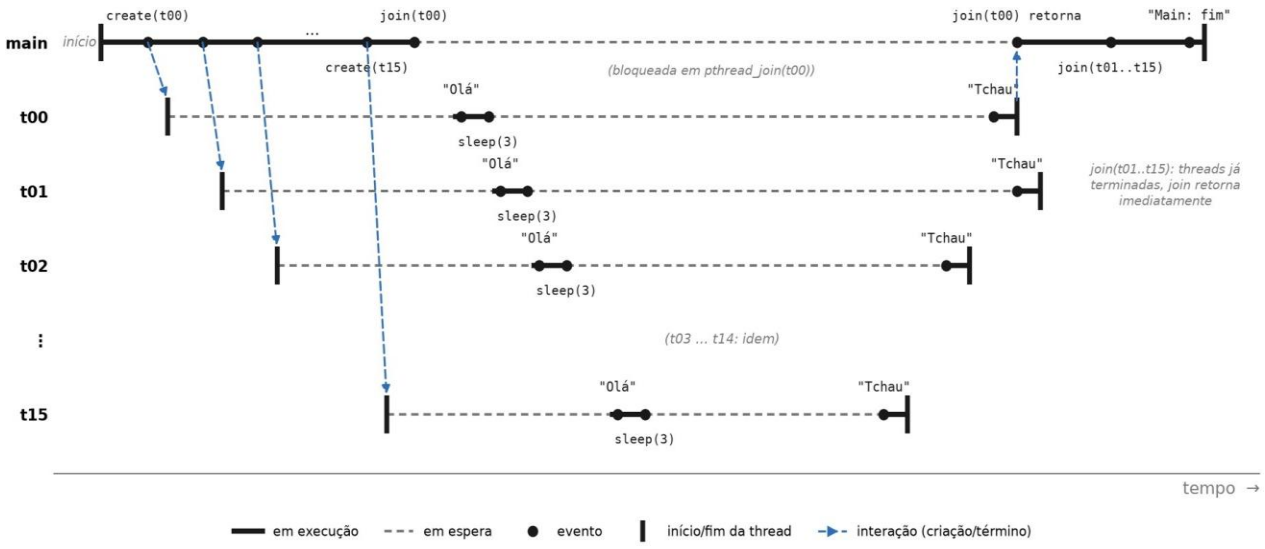
a) Sincronizar: garante que main só continue (e só imprima Main: fim) depois que as threads terminaram;

b) Coletar o valor de retorno da thread (o valor passado a pthread_exit() ou retornado pela função); aqui é NULL, pois o resultado não interessa;

c) Liberar os recursos da thread terminada (pilha e descritor). Uma thread joinable que termina e nunca recebe join permanece como "zumbi", ocupando recursos.

Como os joins são feitos em ordem, main fica bloqueada no join da t00 até que ela termine. Como todas as threads dormem os mesmos 3 s, nesse instante as outras já terminaram (ou estão terminando), e os joins seguintes retornam praticamente de imediato.

2.3 DIAGRAMA DE TEMPO



3 EXERCÍCIO 3: THREAD-PRINT.C

3.1 ANÁLISE DO CÓDIGO

a) A variável x é global. Como as threads compartilham o espaço de endereçamento do processo, existe uma única variável x para as 16 threads;

b) Cada thread executa x++, imprime x, dorme 3 s, executa x++ novamente e imprime x. Sem conflitos, xiria de 1 a 16 nos Oláe de 17 a 32 nos Tchau;

c) No entanto, x++ não é atômico: a operação se divide em três passos (ler x, somar 1 e gravar x). Se duas threads leem o mesmo valor ao mesmo tempo, os dois incrementos produzem o mesmo resultado e um deles se perde, o que caracteriza uma condição de disputa (*race condition*). Como as threads acordam quase juntas depois do sleep(3), os segundos incrementos são os que mais sofrem com isso.

Saída real (primeira execução, trechos repetidos abreviados):

```
Main: criando thread 00
...
Main: fim
t00: Olá!   (x=01)
t04: Olá!   (x=02)
t01: Olá!   (x=03)
...
t15: Olá!   (x=16)
t15: Tchau! (x=17)
t09: Tchau! (x=17)    <- duas threads leram x=16 ao mesmo tempo:
t14: Tchau! (x=18)       um incremento se perdeu
...
t01: Tchau! (x=29)
t04: Tchau! (x=30)
t00: Tchau! (x=31)    <- valor final 31, e não 32
```

Em outra execução, o valor final foi 32 (t00: Tchau! (x=32)). O resultado varia de uma execução para outra justamente por depender da ordem de escalonamento.

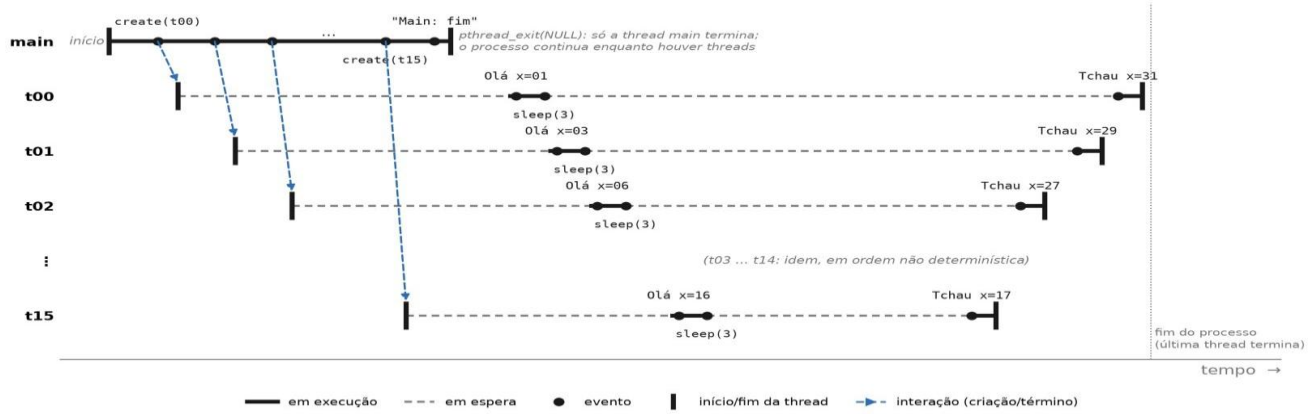
3.2 COMPARAÇÃO COM UM PROGRAMA EQUIVALENTE USANDO FORK

Aspecto	Threads (thread-print.c)	Processos (fork-print.c, Lab 2)
Espaço de endereçamento	Compartilhado: uma única variável x	Separado: cada processo tem sua cópia de x (copy-on-write)
Efeito de x++	Visível a todas as threads	Visível apenas ao próprio processo
Valores impressos	Crescem de 1 até cerca de 32	Cada filho imprimiria x=01 e depois x=02; o pai continuaria com x=0
Valor final	32 (ou menos, com condição de disputa)	0 no pai e 2 em cada filho
Comunicação	Direta, pela memória (exige sincronização)	Exige IPC (pipes, memória compartilhada etc.)

ou seja com as threads existe uma única variável x, o que facilita a cooperação, mas expõe o programa a condições de disputa; para garantir o valor 32 seria preciso um mecanismo de sincronização (mutex ou operações atômicas). Já com fork() não há compartilhamento: o que um processo altera não aparece nos outros, como vimos na atividade 2, em que o pai seguia com x vale 0 mesmo depois de o filho executar x++.

3.3 DIAGRAMA DE TEMPO (COM A EVOLUÇÃO DE X)

Os valores de x são os da primeira execução mostrada acima.



Momento	x
Início (antes de criar as threads)	0
Após o 1º x++ de cada thread (Olá)	1 → 16
Após o 2º x++ de cada thread (Tchau)	17 → 31 (um incremento perdido)
Fim do processo	31 (o esperado seria 32)

4 CÁLCULO DE PI USANDO THREADS (CALCPI.C)

4.1 CÓDIGO-FONTE

O valor de π é aproximado pela série de Leibniz-Gregory, $\pi/4 = \Sigma\ (-1)^n / (2n + 1)$, calculando 10^9 termos com variáveis double. O programa é executado como ./calcpi [num_threads] [num_termos](padrão: 1 thread e 10^9 termos).

a) O intervalo de termos [0, 10^9) é dividido uniformemente entre as N threads: a thread i calcula os termos [i·n/N, (i+1)·n/N), e o resto da divisão, se houver, é distribuído entre as primeiras threads;

b) Cada thread copia as variáveis globais (num_termos, num_threads) para variáveis locais, acumula a soma em uma variável local (na sua pilha) e só no final escreve o resultado em result[i]. Como cada thread escreve em uma posição diferente do vetor, não é necessária sincronização;

c) A thread main espera todas as threads com pthread_join(), soma result[0..N-1], multiplica por 4 e apresenta o resultado.

```
/*
Cálculo de Pi usando threads POSIX (série de Leibniz-Gregory).

    pi/4 = soma_{n=0}^{inf} (-1)^n / (2n + 1)

Uso: ./calcpi [num_threads] [num_termos]
      (padrão: 1 thread, 1 bilhão de termos)

Compilar com gcc -Wall -O2 calcpi.c -o calcpi -lm -lpthread

-----------------------------------------------------------------------------------------------
Funcionamento
-----------------------------------------------------------------------------------------------
O espaço de termos [0, num_termos) é dividido uniformemente entre as N
threads: a thread i calcula os termos [ini_i, fim_i), em que cada bloco tem
num_termos / N termos (o resto da divisão é distribuído, um termo a mais,
entre as primeiras threads). Cada thread soma seus termos em uma variável
LOCAL (na sua própria pilha) e, só ao final, deposita a soma parcial em
result[i]. Como cada thread escreve em uma posição diferente do vetor, não
há condição de disputa e nenhum mecanismo de sincronização é necessário.
O programa principal espera todas as threads (pthread_join), soma os
resultados parciais e multiplica por 4.
*/

#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <math.h>

#define MAX_THREADS 1024
#define TERMOS_PADRAO 1000000000LL    // 10^9 termos

// variáveis globais (apenas lidas pelas threads, exceto result[i])
long long num_termos ;              // total de termos da série
```

```
int             num_threads ;                   // número de threads de cálculo double
                result [MAX_THREADS] ;          // resultados parciais, um por thread


// corpo das threads de cálculo void
*threadBody (void *id)
{
   long i = (long) (intptr_t) id ;              // índice desta thread (0..N-1)


   // cópias locais das variáveis globais (ficam na pilha da thread) long long total =
   num_termos ;
   int             n       = num_threads ;


   // calcula o intervalo de termos [ini, fim) desta thread long long bloco = total / n ;
   long long resto = total % n ;
   long long ini           = i * bloco + (i < resto ? i : resto) ; long long fim    =
   ini + bloco + (i < resto ? 1 : 0) ;


   // soma parcial local; o sinal de (-1)^k depende da paridade de k double soma  = 0.0 ;
   double sinal = (ini % 2 == 0) ? 1.0 : -1.0 ; for (long long k =
   ini; k < fim; k++)
   {
      soma   += sinal / (2.0 * k + 1.0) ; sinal  = -sinal ;
   }


   // deposita o resultado parcial na posição exclusiva desta thread result[i] = soma ;
   return (NULL) ;                              // equivale a pthread_exit (NULL)
}


int main (int argc, char *argv[])
{
   pthread_t thread [MAX_THREADS] ; long i ;
   int status ;


   num_threads = (argc > 1) ? atoi (argv[1]) : 1 ;
   num_termos  = (argc > 2) ? atoll (argv[2]) : TERMOS_PADRAO ;


   if (num_threads < 1 || num_threads > MAX_THREADS || num_termos < 1)
   {
      fprintf (stderr, "Uso: %s [num_threads 1..%d] [num_termos]\n", argv[0],
                       MAX_THREADS) ;
      exit (1) ;
   }


   // cria as threads de cálculo
   for (i = 0; i < num_threads; i++)
   {
      status = pthread_create (&thread[i], NULL, threadBody, (void *) (intptr_t) i) ; if (status)
      {
         perror ("pthread_create") ; exit (1) ;
      }
   }


   // espera o término de todas as threads for (i = 0; i <
   num_threads; i++)
   {
      status = pthread_join (thread[i], NULL) ; if (status)
      {
         perror ("pthread_join") ; exit (1) ;
      }
   }


   // soma os resultados parciais (pi/4) e apresenta o resultado double soma = 0.0 ;
   for (i = 0; i < num_threads; i++) soma += result[i] ;
```

```
  double pi = 4.0 * soma ;


  printf ("threads: %d  termos: %lld\n", num_threads, num_termos) ;
  printf ("pi calculado: %.15f\n", pi) ;
  printf ("erro:         %.3e\n", fabs (pi - M_PI)) ;


  return (0) ;
}
```

Exemplo de execução de calcpi.c com 16 threads

```
$ ./calcpi 16
threads: 16  termos: 1000000000
pi calculado: 3.141592652590205
erro:          9.996e-10
```

4.2 METODOLOGIA

a) Compilação: gcc -Wall -O2 calcpi.c -o calcpi -lm -lpthread;

b) Plataformas: só havia uma máquina disponível. Para obter configurações distintas, a mesma máquina foi restrita a conjuntos diferentes de CPUs por meio da afinidade de processador (SetProcessAffinityMask, herdada pelo processo filho): plataforma A, com 1 CPU lógica (simula uma máquina monoprocessada); plataforma B, com 2 CPUs lógicas em núcleos P distintos (simula uma máquina dual); e plataforma C, com as 16 CPUs lógicas do i7-13620H;

c) Foi medido o tempo real (*wall-clock*, equivalente ao real do comando time) de cada execução completa do programa. A coleta foi automatizada pelo script scripts/bench.py;

d) Foram feitas 5 medições para cada combinação (N = 1, 2, 4, 8, 16 e 32), calculando a média e o coeficiente de variação (CV = desvio-padrão amostral / média). Séries com CV acima de 5% foram refeitas. Antes de cada plataforma foram feitas 8 execuções de aquecimento, pois uma primeira rodada mostrou que, com a CPU "fria", a frequência de clock variava durante a série (o mesmo teste levava de 1,33 s a 1,0 s);

e) Antes das medições, a carga da máquina era de cerca de 12 a 14% de uso de CPU (processos em segundo plano), verificada pelo contador de desempenho do Windows (equivalente ao top). Os dados brutos estão no arquivo medicoes.csv.

4.3 TEMPOS DE EXECUÇÃO

Tempos de execução na plataforma A (1 CPU, monoprocessada)

Threads	t1 (s)	t2 (s)	t3 (s)	t4 (s)	t5 (s)	Média (s)	CV
1	1,025	0,960	0,951	0,931	0,950	**0,964**	3,72%
2	0,973	0,974	0,960	0,976	0,958	**0,968**	0,88%
4	0,949	0,947	0,929	0,952	0,942	**0,944**	0,97%
8	0,947	0,950	0,948	0,951	0,928	**0,945**	1,02%
16	0,957	0,925	0,952	0,986	0,949	**0,954**	2,29%
32	0,948	0,933	0,924	0,944	0,955	**0,941**	1,29%

Tempos de execução na plataforma B (2 CPUs, dual)

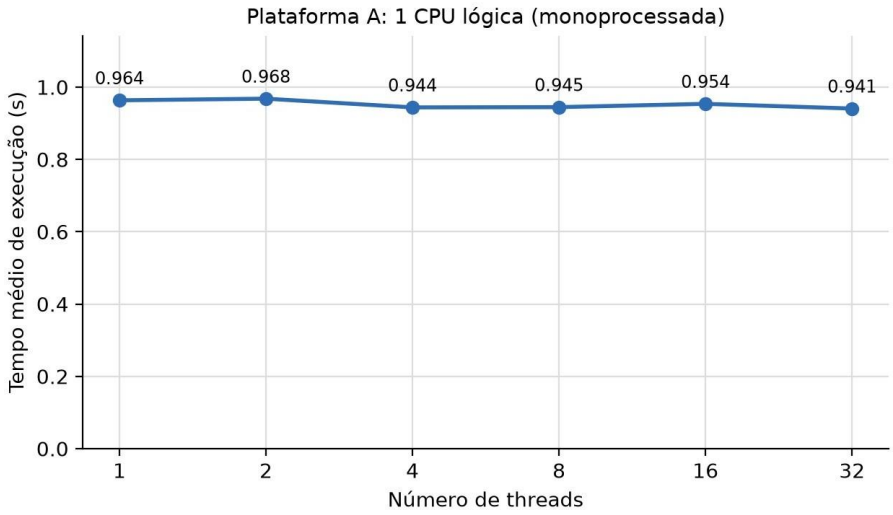
Threads	t1 (s)	t2 (s)	t3 (s)	t4 (s)	t5 (s)	Média (s)	CV
1	0,931	0,945	0,959	0,938	0,973	**0,949**	1,79%
2	0,488	0,472	0,480	0,510	0,490	**0,488**	2,89%
4	0,481	0,473	0,475	0,482	0,484	**0,479**	0,95%
8	0,524	0,476	0,482	0,487	0,484	**0,491**	3,86%
16	0,474	0,495	0,471	0,483	0,488	**0,482**	2,05%
32	0,479	0,483	0,475	0,478	0,503	**0,483**	2,35%

Tempos de execução na plataforma C (16 CPUs)

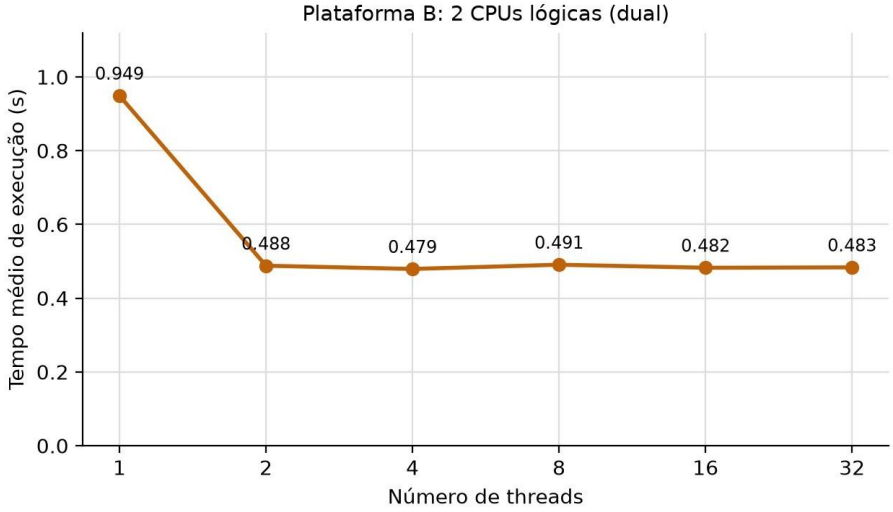
Threads	t1 (s)	t2 (s)	t3 (s)	t4 (s)	t5 (s)	Média (s)	CV
1	0,925	0,922	0,927	0,925	0,930	**0,926**	0,30%
2	0,479	0,460	0,463	0,465	0,467	**0,467**	1,54%
4	0,237	0,234	0,234	0,236	0,247	**0,238**	2,20%
8	0,178	0,189	0,199	0,194	0,181	**0,188**	4,74%
16	0,152	0,147	0,149	0,149	0,151	**0,149**	1,10%
32	0,155	0,150	0,153	0,150	0,154	**0,152**	1,36%

Todos os coeficientes de variação ficaram abaixo de 5%.

4.4 GRÁFICOS DE TEMPO MÉDIO POR PLATAFORMA

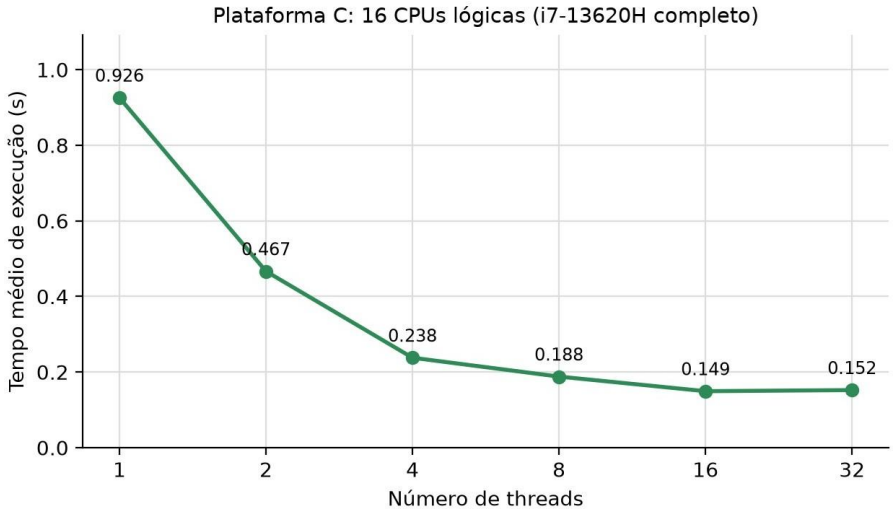


Plataforma A (1 CPU): a curva é plana (cerca de 0,94 a 0,97 s para qualquer N). Com uma única CPU as threads não executam em paralelo: o escalonador as alterna (*time-sharing*) e o trabalho total, 10^9 termos, é o mesmo. Assim, dividir o cálculo em mais threads não reduz o tempo. Também não o aumenta de forma perceptível, pois o custo de criar até 32 threads e de trocar de contexto entre elas é desprezível diante de cerca de 1 s de cálculo.



Plataforma B (2 CPUs): o tempo cai pela metade de 1 para 2 threads (0,949 s para 0,488 s, *speedup* de 1,94), quase o ideal. A partir de 2 threads a curva fica plana

(cerca de 0,48 s): as duas CPUs já estão totalmente ocupadas e as threads adicionais apenas dividem o mesmo tempo de processador.



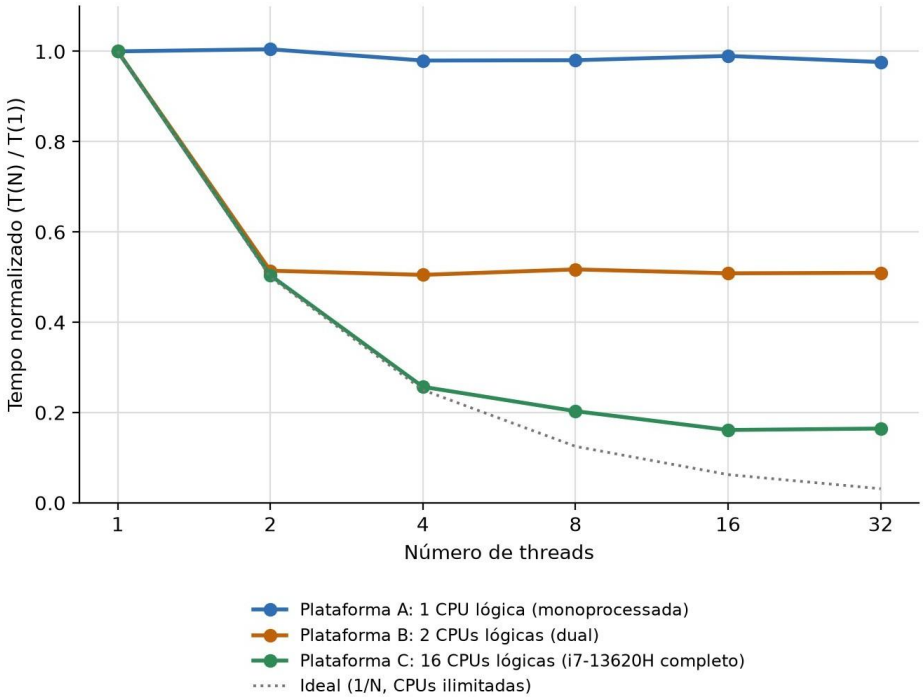
Plataforma C (16 CPUs): o tempo cai quase idealmente até 4 threads (0,926 s, 0,467 s e 0,238 s; *speedups* de 1,98 e 3,89). Depois o ganho diminui: 0,188 s com 8 threads (*speedup* de 4,9) e 0,149 s com 16 threads (*speedup* de 6,2, o melhor resultado). Com 32 threads o tempo se mantém (0,152 s), pois não há mais CPUs livres. A queda abaixo do ideal a partir de 8 threads tem duas causas ligadas ao processador híbrido:

a) Hyper-Threading: as 16 CPUs lógicas correspondem a apenas 10 núcleos físicos. Duas threads no mesmo núcleo P disputam as mesmas unidades de ponto flutuante, e o laço do cálculo é limitado justamente por divisões em `double`; por isso, a segunda thread de um núcleo rende bem menos que um núcleo inteiro;

b) Núcleos E mais lentos e divisão estática: cada thread recebe a mesma quantidade de termos, e as que executam nos núcleos de eficiência (de menor frequência) demoram mais. Como `main` precisa esperar todas as threads (join), o tempo total é ditado pela mais lenta.

Além disso, há uma parcela fixa não paralelizável (iniciar o processo e criar as threads, alguns milissegundos), que limita o ganho, conforme a lei de Amdahl.

4.5 GRÁFICO CONSOLIDADO COM TEMPOS NORMALIZADOS



Tempos normalizados (T(N) / T(1))

Threads	A (1 CPU)	B (2 CPUs)	C (16 CPUs)	Ideal (1/N)
1	1,000	1,000	1,000	1,000
2	1,005	0,514	0,504	0,500
4	0,980	0,505	0,257	0,250
8	0,980	0,517	0,203	0,125
16	0,990	0,508	0,161	0,063
32	0,976	0,509	0,164	0,031

Normalizar pelo tempo com 1 thread elimina as diferenças absolutas entre as plataformas e mostra apenas o quanto cada uma aproveita as threads. As três curvas partem de 1,0, acompanham a curva ideal enquanto o número de threads é menor ou igual ao número de CPUs e "saturam" quando N ultrapassa o número de CPUs disponíveis: em 1,0 na plataforma A, em cerca de 0,5 na B e em cerca de 0,16 na C.

Portanto, o fator que limita o ganho é o número de CPUs, e não o número de threads. Criar mais threads que CPUs não acelera um cálculo limitado por CPU (*CPU-*

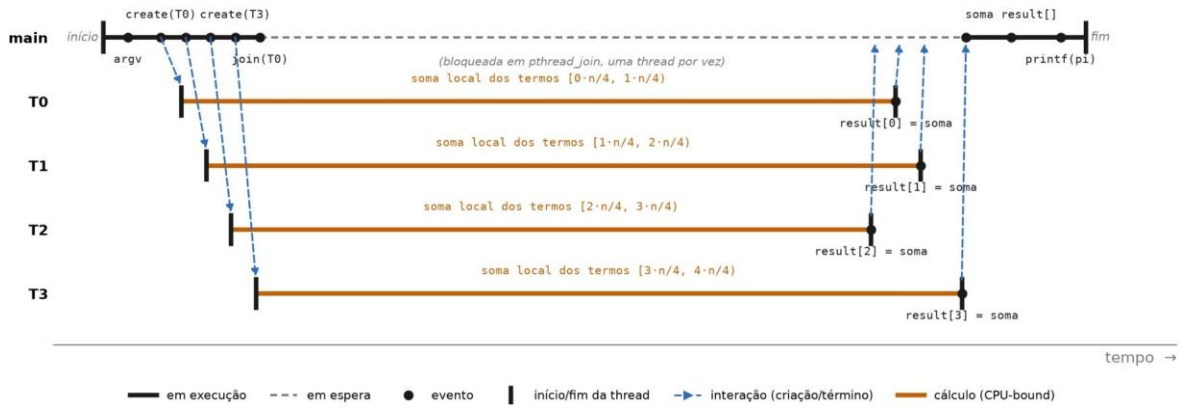
bound); também não o prejudica de forma mensurável (as curvas ficam planas, sem subir), pois o custo de criar e escalonar threads é pequeno diante do cálculo.

A plataforma C se afasta do ideal a partir de 8 threads (0,203 em vez de 0,125), pelos motivos discutidos anteriormente: Hyper-Threading, núcleos E mais lentos e divisão estática do trabalho. Uma divisão dinâmica, com blocos menores distribuídos sob demanda, compensaria parte da diferença entre os núcleos P e E.

Como limitação do experimento, as três "plataformas" são a mesma máquina com afinidade restrita, e não computadores diferentes. O clock e a arquitetura são os mesmos, de modo que o efeito observado é apenas o do número de CPUs. Além disso, o clock dinâmico (turbo) do notebook varia conforme a temperatura e a quantidade de núcleos ativos, o que exigiu o aquecimento prévio e pode favorecer levemente as execuções com poucas threads.

4.6 DIAGRAMA DE TEMPO DA EXECUÇÃO

Diagrama simplificado da execução com N = 4 threads:



4.7 QUESTÕES

1. As threads implementadas são preemptivas ou cooperativas? Explique sua resposta.

São preemptivas. Nenhuma thread do calcpi cede a CPU voluntariamente: o laço de cálculo não chama sched_yield(), sleep() nem qualquer chamada bloqueante, e mesmo assim todas progridem quando há mais threads que CPUs. Na plataforma A (1 CPU) com 32 threads, se as threads fossem cooperativas, a primeira

a executar ocuparia a CPU até terminar seu bloco e só então a próxima seria executada; o sistema ficaria sem resposta durante o cálculo. Na prática, o tempo total é o mesmo e o restante do sistema (inclusive o próprio script de medição) continua responsivo. Isso só é possível porque o núcleo do sistema operacional interrompe a thread em execução ao final de seu *quantum* (por meio da interrupção do temporizador) e escalona outra. As threads POSIX criadas são threads de núcleo escalonadas pelo Windows, que usa escalonamento preemptivo por prioridades; o mesmo vale para o Linux.

2. Que modelo de threads o sistema operacional usado implementa (N:1, 1:1 ou N:M)? Como isso pode ser deduzido a partir dos experimentos?

O modelo é 1:1: cada pthread_create() gera uma thread de núcleo, e o escalonador do sistema operacional pode colocá-la em qualquer CPU. No Windows, a biblioteca *winpthreads* associa cada pthread_t a uma thread nativa (CreateThread); no Linux, a NPTL segue a mesma ideia.

Dá para deduzir isso dos experimentos. No modelo N:1, todas as threads de usuário do processo ficam sobre uma só thread de núcleo, então o processo usaria apenas uma CPU por vez e as curvas das plataformas B e C ficariam iguais à da A (planas, em torno de 0,95 s), o que não aconteceu. O que observamos foi o tempo diminuir na proporção do número de CPUs (cerca de metade com 2 CPUs e em torno de 1/6 com 16), ou seja, threads de um mesmo processo rodaram ao mesmo tempo em núcleos diferentes, algo que só é possível se o núcleo do sistema enxergar e escalonar cada thread separadamente.

Isso descarta o modelo N:1, mas sozinho não distinguiria o 1:1 do N:M, que também usa várias CPUs. O que indica o modelo 1:1 é que o ganho acompanha o número de CPUs até o limite de CPUs disponíveis para qualquer N, sem indício de um número fixo de threads de núcleo menor que N. Somado à implementação conhecida da biblioteca (uma thread nativa por pthread_t), conclui-se que o modelo é 1:1.